

Automated Research Paper Summarizer and Critique System

Computer DLOC Lab I CE509 Agentic AI

Domain: Education and Learning

Problem Statement Number: 5

Student Name: Ghosh Chetan Devdulal

Roll Number: 5

Email: chetan26mtce@student.pillai.edu.in

Semester: 1

Submission Date: 06 October 2026

Project Report

1 Objective

The objective is to build a local application that helps a student review several research papers in a consistent way. Specialized agents search literature, validate inputs, select summary sentences, identify methods and results, and screen methodological reporting. The application compares evidence across papers and prepares review questions. Source sentence references make outputs traceable. The implemented system uses extractive rules and does not perform LLM reasoning or replace expert peer review.

2 Work Done

2 1 Problem understanding

Reading several papers manually makes it difficult to keep track of methods, datasets, results, and limitations. The project organizes these fields into one workspace. The assignment was decomposed into small responsibilities so each agent returns a clear output for the next stage.

- Separate source evidence from screening questions so that an unreported detail is not presented as a proven weakness.
- Support paper text, selectable-text PDF files, and JSON reading lists while preserving titles and bibliographic details.
- Compare result statements without ranking studies that use different metrics or datasets.

2 2 Task decomposition

The implementation separates literature discovery, ingestion, summarization, evidence extraction, critique, comparison, and report assembly. Users choose which papers to add; the system does not automatically fetch full text from publisher pages. Uploaded text is treated as data and no instructions inside it are executed.

2 Work Done continued

2.3 Agent roles and responsibilities

Table 1 Agent responsibilities

Agent	Responsibility
LiteratureAgent	Search Crossref journal metadata and return titles, authors, DOI, year, and available abstracts.
IngestionAgent	Validate 1 to 10 papers, require titles and usable text, and assign P1, P2 identifiers.
SummaryAgent	Rank source sentences using normalized word frequency and select up to three in source order.
MethodologyAgent	Find sentences about methods, datasets, results, and limitations using pattern rules.
CritiqueAgent	Detect baseline, uncertainty, reproducibility, and validation reporting signals; raise follow-up questions.
ComparisonAgent	Build a side-by-side method, data, and result matrix without ranking incompatible results.
Review coordinator	Assemble agent outputs, deduplicate review questions, and create exportable reports.

2.4 Communication design

Agents communicate through Python dictionaries and lists in a sequential coordinator function. The browser sends a JSON request to the local server. After input validation, the coordinator passes each paper to the summary and methodology agents, passes extracted evidence to the critique agent, and passes all processed papers to the comparison agent. The server returns one JSON report. There is no inter-agent chat service or parallel execution.

2 Work Done continued

2.5 Tools and implementation

- Python standard-library ThreadingHTTPServer exposes the local web interface and analysis endpoints.
- HTML, CSS, and JavaScript implement search, paper entry, imports, results, and download controls.
- pypdf extracts selectable text from PDFs. Scanned PDFs need OCR before import.
- Crossref REST API supplies search metadata. Requests use a fixed endpoint, a user-agent header, and a timeout.
- Python unittest verifies analysis behavior without requiring a live literature service.

2.6 Architecture and workflow

Literature search or PDF TXT JSON input
Paper editor and user verification
IngestionAgent validates paper list
SummaryAgent and MethodologyAgent
CritiqueAgent evaluates reporting signals
ComparisonAgent and review coordinator
Browser report and Markdown JSON export

Figure 1 Workflow from input to output with downward handoffs

2.7 Operating limits

The server binds to 127.0.0.1 on port 8001. PDF uploads are limited to 8 MB and 100 pages. Each paper is limited to 250,000 text characters. Text and PDFs are processed in memory. Search queries are sent to Crossref, while imported paper text remains local.

3 Results and Output

3 1 Running system

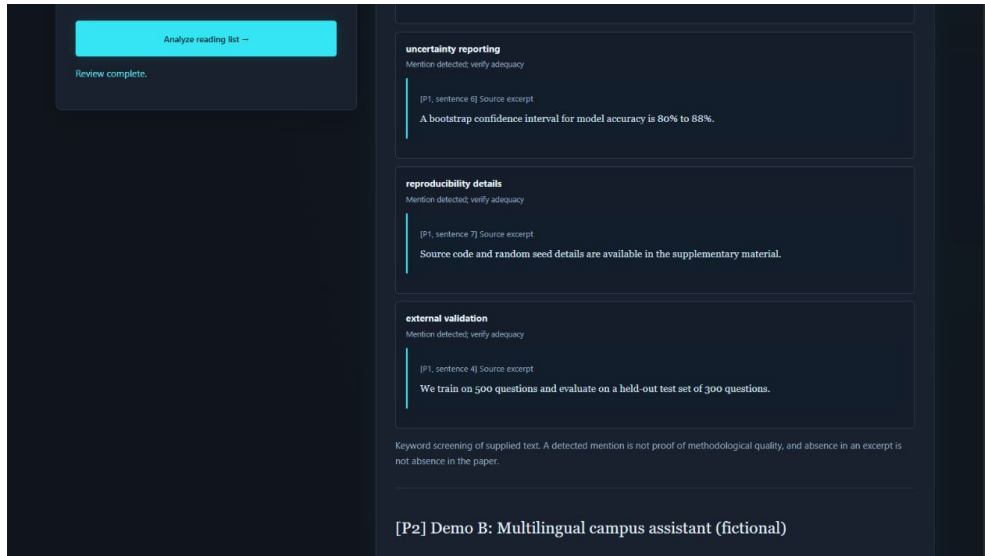


Figure 2 Running application after analysis of the fictional reading list

The interface displayed two papers, a completed agent trace, summary excerpts, and enabled Markdown and JSON export controls. The visible results were produced by clicking Load fictional demo and Analyze reading list in the local application.

3 2 Sample input and corresponding output

The demonstration uses two fictional papers. P1 contains 809 characters and nine parsed sentences about a retrieval-assisted campus FAQ system. P2 contains 586 characters and eight parsed sentences about a multilingual campus assistant. These are demonstration inputs, not published research findings.

Table 2 Selected source inputs and extracted outputs

Paper	Input evidence	Output
P1	800 questions; 500 train, 300 test; accuracy 84% versus 72%.	Summary sentences 1, 4, 6; baseline comparison extracted.
P2	1,200 bilingual pairs; 240 test questions; F1 0.81, recall 0.86.	Summary sentences 1, 2, 6; numerical results extracted.

3 Results and Output continued

3.3 Agent interaction trace

IngestionAgent validates inputs → SummaryAgent selects sentences → MethodologyAgent extracts evidence → CritiqueAgent screens reporting → ComparisonAgent builds the matrix → coordinator assembles the review.

For P2, the critique agent did not detect baseline comparison, uncertainty reporting, or reproducibility details. It generated three corresponding questions for checking the full paper and supplementary material. Both examples contain a held-out split, which triggers the current validation rule; this is a reporting signal rather than proof of independent external validation.

3.4 Evaluation

Table 3 Verification results

Check	Observed result
Automated unit tests	6 of 6 passed on 6 October 2026.
Source grounding	Selected summary sentences occur in source text.
Metadata mapping	Mocked mapping and markup removal passed.
Input handling	Invalid papers and short queries rejected.
Local API and PDF checks	HTTP 200, analysis, Markdown, and PDF extraction passed.
JavaScript syntax	Node syntax check passed.

Tests verify functional behavior, not scientific quality. No expert quality, precision, recall, or latency benchmark was measured. Demo accuracy and F1 values belong to fictional input papers, not this analyzer.

3.5 Code repository and export

Repository name: chetan-research-paper-analyzer

GitHub URL: <https://github.com/chetan-sys-ops/chetan-research-paper-analyzer/tree/main>

JSON and Markdown exports include evidence, review questions, and source references.

4 Observations and Learning

- Separate agents made the pipeline easy to inspect. Sentence references linked summaries and evidence to the supplied text.
- Frequency-based summaries were reproducible, but did not always select the most important numerical result. The methodology panel provided additional result evidence.
- Missing abstracts were handled by allowing users to add full text. Input validation prevented metadata-only records from being analyzed.
- Keyword patterns could overstate validation: a held-out split matched the external-validation category. The output therefore needs human checking and a more precise rule in future versions.
- PDF extraction can change column order. The editor allows review before analysis; OCR and reliable layout reconstruction remain outside the current scope.

The main learning was to preserve evidence, document uncertainty, and test each handoff separately. Functional tests establish expected behavior for examples, while evaluation of scientific quality needs a separate labelled benchmark and expert review.

5 Conclusion

The project provides a working local workflow for literature discovery and paper review. It imports paper text, creates source-linked extractive summaries, identifies methodology evidence, and compares results. Six automated tests passed, and local API and PDF extraction checks succeeded. The critical review helps readers identify details to verify. Future work should improve extraction rules and evaluate output quality using expert-labelled research papers.

6 References

- [1] CE509 Agentic AI Laboratory, Computer DLOC Lab I Instructions Set, supplied course document, pp. 1-2.
- [2] CE509 Agentic AI Laboratory, Problem Statements, supplied course document, Problem Statement 5, p. 1.
- [3] Crossref, REST API documentation. <https://www.crossref.org/documentation/retrieve-metadata/rest-api/>
- [4] Python Software Foundation, Python 3 documentation. <https://docs.python.org/3/>
- [5] pypdf contributors, pypdf documentation. <https://pypdf.readthedocs.io/>

Appendix A Prompts and Selected Code

A 1 User inputs and important development prompt

Development request: Build the Automated Research Paper Summarizer and Critique System for Ghosh Chetan Devdulal in the selected project folder.

Example literature query: retrieval augmented question answering. The query is sent to Crossref as search data. The analysis agents do not use runtime LLM system prompts or prompt templates.

A 2 Selected coordinator code

```
clean = IngestionAgent().run(papers)
result = []
for p in clean:
    evidence = MethodologyAgent().run(p)
    result.append(
        {k: v for k, v in p.items() if k != 'text'}
        | SummaryAgent().run(p)
        | {'evidence': evidence,
          'critique': CritiqueAgent().run(p, evidence),
          'sentence_count': len(sentences(p['text']))}
    )
comparison = ComparisonAgent().run(result)
```

Listing 1 Selected excerpt from agents.py showing structured agent handoffs

A 3 Setup and reproduction

- Install Python 3.10 or later. Run setup.bat once to install pypdf, then start.bat to launch the server.
- Open <http://127.0.0.1:8001/> and run the fictional demonstration. Import verified paper text for a real study.
- Run `python -m unittest discover -s tests -v` from the project folder to repeat the six automated tests.

Appendix B Additional Screenshots

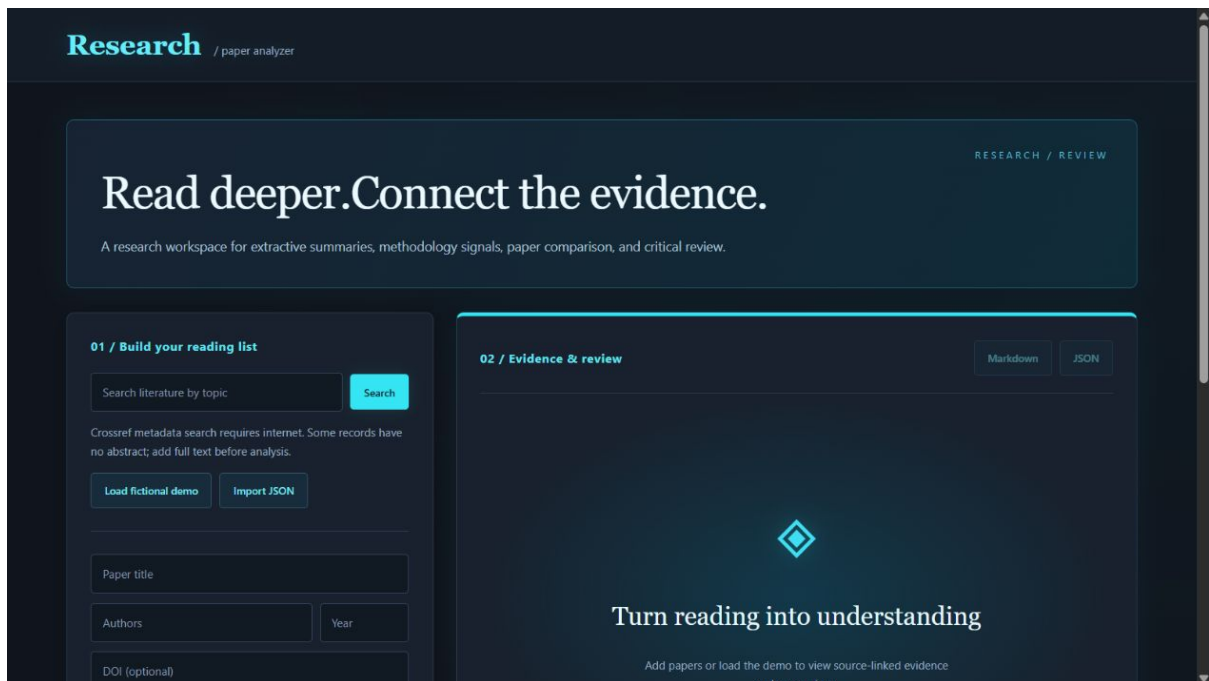


Figure 3 Initial paper editor and evidence workspace

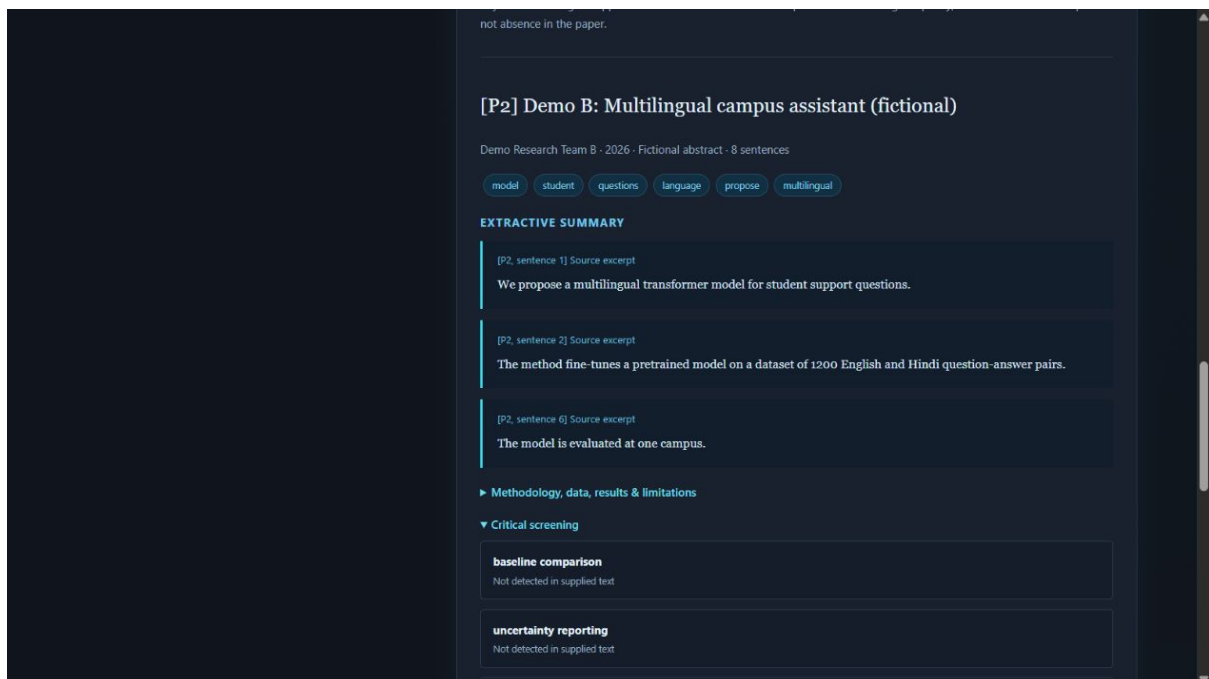


Figure 4 Detailed results showing source excerpts and the validation screening limitation